

Tarea UT4 Optimización y documentación

Abraham Quintana Micó

1º DAW Semi B

Tarea 1: Refactorización

Estado inicial del proyecto:

```
package com.mycompany.deposito;

public class Cuenta {

    String nombre;
    String cuenta;
    double saldo;
    double tipoInterés;

    public Cuenta() {
    }

    public Cuenta(String nom, String cue, double sal, double tipo) {
        nombre = nom;
        cuenta = cue;
        saldo = sal;
    }

    public double estado() {
        return saldo;
    }

    public void retirar(double cantidad) {
        try {
            System.out.println("Retiro en cuenta:" + cantidad);
            if (cantidad <= 0) {
                throw new Exception("No se puede retirar una cantidad negativa");
            }
            if (estado() < cantidad) {
                throw new Exception("No se hay suficiente saldo");
            }
            saldo = saldo - cantidad;
        } catch (Exception e) {
            System.out.println(e);
        }
    }

    public void ingresar(double cantidad) {
        try {
            System.out.println("Ingreso en cuenta:" + cantidad);
            if (cantidad < 0) {
                throw new Exception("No se puede ingresar una cantidad negativa");
            }
            saldo = saldo + cantidad;
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

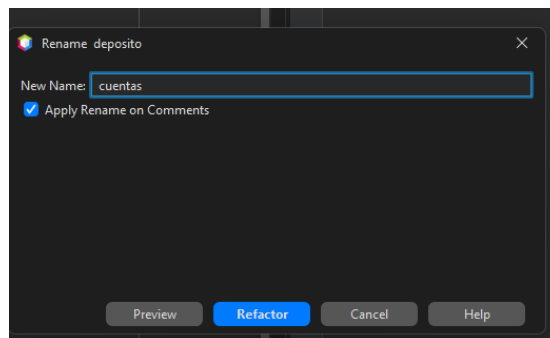
package com.mycompany.deposito;

public class Main {

    public static void main(String[] args) {
        Cuenta miCuenta;
        double saldoActual;
        miCuenta = new Cuenta(nom: "Antonio López", cue: "1000-2365-85-1230456789", sal: 2500, tip: 0.05);
        saldoActual = miCuenta.estado();
        System.out.println("El saldo actual es " + saldoActual);

        // Operaciones de ingreso y retiro
        miCuenta.retirar(cantidad:2300);
        miCuenta.ingresar(cantidad:685);
        saldoActual = miCuenta.estado();
        System.out.println("El saldo actual es " + saldoActual);
    }
}
```

1. Las clases deberán formar parte del paquete “cuentas”.



```
package com.mycompany.cuentas;

public class CCuenta {

    String nombre;
    String cuenta;
    double saldo;
    double tipoInterés;

    public CCuenta() {
    }

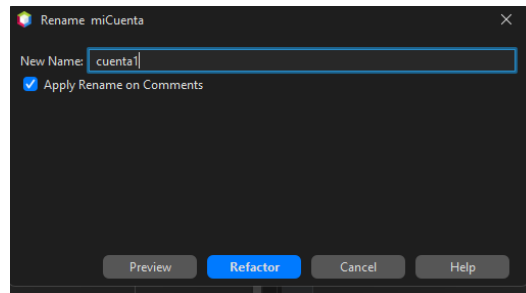
    package com.mycompany.cuentas;

    public class Main {

        public static void main(String[] args) {
            Cuenta miCuenta;
            double saldoActual;
            miCuenta = new Cuenta(nom: "Antonio López", cue: "1000-2365-85-1230456789", sal: 2500, tip: 0.05);
            saldoActual = miCuenta.estado();
            System.out.println("El saldo actual es " + saldoActual);

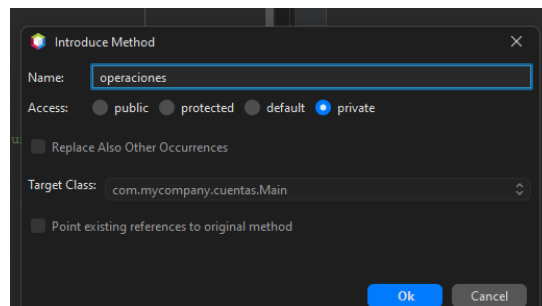
            // Operaciones de ingreso y retiro
            miCuenta.retirar(cantidad:2300);
            miCuenta.ingresar(cantidad:685);
            saldoActual = miCuenta.estado();
            System.out.println("El saldo actual es " + saldoActual);
        }
    }
```

2. Cambiar el nombre de la variable "**miCuenta**" por "**cuenta1**".



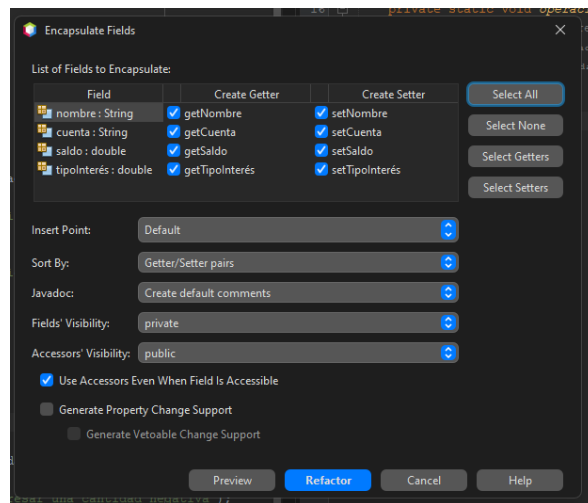
```
1 package com.mycompany.cuentas;
2
3 public class Main {
4
5     public static void main(String[] args) {
6         CCuenta cuenta1;
7         double saldoActual;
8         cuenta1 = new CCuenta("Antonio López", "1000-2365-85-1230456789", 2500, 2000);
9         saldoActual = cuenta1.estado();
10        System.out.println("El saldo actual es " + saldoActual);
11        // Operaciones de ingreso y retiro
12        cuenta1.retirar(cantidad:2300);
13        cuenta1.ingresar(cantidad:685);
14        saldoActual = cuenta1.estado();
15        System.out.println("El saldo actual es " + saldoActual);
16    }
17 }
```

3. Introducir el método "**operaciones**", que englobe las dos sentencias de la clase Main que realizan operaciones de saldo. Seleccionar líneas, botón derecho del ratón y utilizar Refactor, Introduce y Method...



```
Source History
Main.java x
1 package com.mycompany.cuentas;
2
3 public class Main {
4
5     public static void main(String[] args) {
6         CCuenta cuenta1;
7         double saldoActual;
8         cuenta1 = new CCuenta("Antonio López", "1000-2365-85-1230456789", 2500, 2000);
9         saldoActual = cuenta1.estado();
10        System.out.println("El saldo actual es " + saldoActual);
11        operaciones(cuenta1);
12        saldoActual = cuenta1.estado();
13        System.out.println("El saldo actual es " + saldoActual);
14    }
15
16     private static void operaciones(CCuenta cuenta1) {
17         // Operaciones de ingreso y retiro
18         cuenta1.retirar(cantidad:2300);
19         cuenta1.ingresar(cantidad:685);
20     }
21 }
22 }
```

4. Encapsular los atributos de la clase **CCuenta**. Ir a la clase CCuenta, botón derecho del ratón y utilizar Refactor, Encapsulate Fields, Select All y Refactor.



Atributos encapsulados en imagen de la izquierda, getter & setters en la imagen derecha.

```
package com.mycompany.cuentas;

public class CCuenta {

    private String nombre;
    private String cuenta;
    private double saldo;
    private double tipoInterés;

}
```

```
/**
 * @return the nombre
 */
public String getNombre() {
    return nombre;
}

/**
 * @param nombre the nombre to set
 */
public void setNombre(String nombre) {
    this.nombre = nombre;
}

/**
 * @return the cuenta
 */
public String getCuenta() {
    return cuenta;
}

/**
 * @param cuenta the cuenta to set
 */
public void setCuenta(String cuenta) {
    this.cuenta = cuenta;
}

/**
 * @return the saldo
 */
public double getSaldo() {
    return saldo;
}

/**
 * @param saldo the saldo to set
 */
public void setSaldo(double saldo) {
    this.saldo = saldo;
}

/**
 * @return the tipoInterés
 */
public double getTipoInterés() {
    return tipoInterés;
}

/**
 * @param tipoInterés the tipoInterés to set
 */
public void setTipoInterés(double tipoInterés) {
    this.tipoInterés = tipoInterés;
}

}
```

Tarea 2: GIT

Creación del repositorio en git (local):

Configuración del repositorio y se añade los ficheros del proyecto:

```
PS C:\Users\abrah\Documents\NetBeansProjects\Deposito> git init
Initialized empty Git repository in C:/Users/abrah/Documents/NetBeansProjects/Deposito/.git/
PS C:\Users\abrah\Documents\NetBeansProjects\Deposito> git commit -m "first commit"
On branch main

Initial commit

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    pom.xml
    src/

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\abrah\Documents\NetBeansProjects\Deposito> git branch -M main
PS C:\Users\abrah\Documents\NetBeansProjects\Deposito> git remote add origin https://github.com/AbrahamQM/Tarea_EntornosDeDesarrollo.git
PS C:\Users\abrah\Documents\NetBeansProjects\Deposito> git add .
warning: in the working copy of 'pom.xml', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/main/java/com/mycompany/cuentas/CCuenta.java', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/main/java/com/mycompany/cuentas/Main.java', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\abrah\Documents\NetBeansProjects\Deposito> git commit -m "added project files"
```

1º Commit

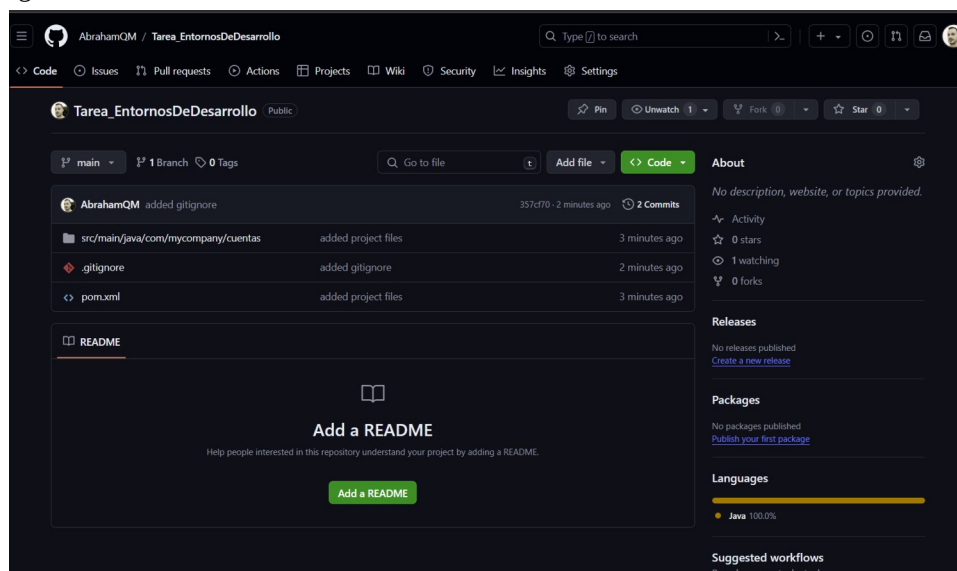
```
PS C:\Users\abrah\Documents\NetBeansProjects\Deposito> git commit -m "added project files"
[main (root-commit) bb51287] added project files
 3 files changed, 140 insertions(+)
 create mode 100644 pom.xml
 create mode 100644 src/main/java/com/mycompany/cuentas/CCuenta.java
 create mode 100644 src/main/java/com/mycompany/cuentas/Main.java
PS C:\Users\abrah\Documents\NetBeansProjects\Deposito>
```

2º Commit añadiendo el .gitignore personalizado para java y subida del repositorio local al remoto.

```
PS C:\Users\abrah\Documents\NetBeansProjects\Deposito> git commit -m "added gitignore"
[main 357cf70] added gitignore
 1 file changed, 24 insertions(+)
 create mode 100644 .gitignore
PS C:\Users\abrah\Documents\NetBeansProjects\Deposito> git push -u origin main
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 12 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (14/14), 2.17 KiB | 1.08 MiB/s, done.
Total 14 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/AbrahamQM/Tarea_EntornosDeDesarrollo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Link al repositorio: https://github.com/AbrahamQM/Tarea_EntornosDeDesarrollo.git

Repositorio en github:



Git log:

```
PS C:\Users\abrah\Documents\NetBeansProjects\Deposito>
PS C:\Users\abrah\Documents\NetBeansProjects\Deposito> git log
commit 357cf70252ba0926e869ed254a1d83efb7dd0632 (HEAD -> main, origin/main)
Author: Abraham Quintana <abrahamqm87@gmail.com>
Date:   Fri May 24 19:01:09 2024 +0100

    added gitignore

commit bb5128737c853f34b7dc09bcf31ce0acdccffdd3
Author: Abraham Quintana <abrahamqm87@gmail.com>
Date:   Fri May 24 18:59:32 2024 +0100

    added project files
PS C:\Users\abrah\Documents\NetBeansProjects\Deposito> |
```

Tarea 3: JAVADOC

1. Insertar comentarios JavaDoc en la clase CCuenta.

```
1 package com.mycompany.cuentas;
2
3 /**
4  * Clase principal que contiene el método main para ejecutar la aplicación de gestión de cuentas bancarias.
5  *
6  * <see <a href="https://docs.oracle.com/en/java/">Java Documentation</a>
7  * <since 2024-05-24
8  * <version 1.0
9  * <author Abraham Quintana
10  */
11 public class Main {
12
13     /**
14      * Método principal que ejecuta la aplicación.
15      *
16      * @param args los argumentos de la línea de comandos (no utilizados en esta aplicación).
17      */
18     public static void main(String[] args) {
19         CCuenta cuenta1;
20         double saldoActual;
21         cuenta1 = new CCuenta("Antonio López", "cuc", "1000-2365-65-1230456789", 2500, 0);
22         saldoActual = cuenta1.estado();
23         System.out.println("El saldo actual es " + saldoActual);
24         operaciones(cuenta1);
25         saldoActual = cuenta1.estado();
26         System.out.println("El saldo actual es " + saldoActual);
27     }
28
29     /**
30      * Método que realiza operaciones de retiro e ingreso en una cuenta bancaria.
31      *
32      * @param cuenta1 la cuenta bancaria sobre la que se realizarán las operaciones.
33      */
34     private static void operaciones(CCuenta cuenta1) {
35         // Operaciones de ingreso y retiro
36         cuenta1.retirar(2300);
37         cuenta1.ingresar(685);
38     }
39 }
40
```

```
1 package com.mycompany.cuentas;
2
3 /**
4  * Clase que representa una cuenta bancaria.
5  * <p>
6  * Esta clase permite realizar operaciones básicas de una cuenta bancaria como ingresar
7  * y retirar dinero, así como consultar el saldo actual.
8  * </p>
9  *
10  * <see <a href="https://docs.oracle.com/en/java/">Java Documentation</a>
11  * <since 2024-05-24
12  * <version 1.0
13  * <author Abraham Quintana
14  */
15 public class CCuenta {
16
17     private String nombre;
18     private String cuenta;
19     private double saldo;
20     private double tipoInterés;
21
22     /**
23      * Constructor por defecto.
24      */
25     public CCuenta() {
26     }
27
28     /**
29      * Constructor con parámetros para inicializar una cuenta bancaria.
30      *
31      * @param nom el nombre del titular de la cuenta.
32      * @param cue el número de la cuenta bancaria.
33      * @param sal el saldo inicial de la cuenta.
34      * @param tipo el tipo de interés de la cuenta.
35      */
36     public CCuenta(String nom, String cue, double sal, double tipo) {
37         nombre = nom;
38         cuenta = cue;
39         saldo = sal;
40         tipoInterés = tipo;
41     }
42
43     /**
44      * Método que devuelve el saldo actual de la cuenta.
45      *
46      * @return el saldo actual de la cuenta.
47      */
48     public double estado() {
49         return getSaldo();
50     }
51
52     /**
53      * Método para retirar una cantidad de dinero de la cuenta.
54      *
55      * @param cantidad la cantidad a retirar.
56      */
57     public void retirar(double cantidad) {
58         try {
59             System.out.println("Retiro en cuenta:" + cantidad);
60             if (cantidad <= 0) {
61                 throw new Exception("No se puede retirar una cantidad negativa");
62             }
63             if (estado() < cantidad) {
64                 throw new Exception("No hay suficiente saldo");
65             }
66             setSaldo(getSaldo() - cantidad);
67         } catch (Exception e) {
68             System.out.println(e);
69         }
70     }
71
72     /**
73      * Método para ingresar una cantidad de dinero en la cuenta.
74      *
75      * @param cantidad la cantidad a ingresar.
76      */
77     public void ingresar(double cantidad) {
78         try {
79             System.out.println("Ingreso en cuenta:" + cantidad);
80             if (cantidad < 0) {
81                 throw new Exception("No se puede ingresar una cantidad negativa");
82             }
83             setSaldo(getSaldo() + cantidad);
84         } catch (Exception e) {
85             System.out.println(e);
86         }
87     }
88
89     /**
90      * Obtiene el nombre del titular de la cuenta.
91      *
92      * @return el nombre del titular.
93      */
94     public String getNombre() {
95         return nombre;
96     }
97
98     /**
99      * Establece el nombre del titular de la cuenta.
100      *
101      * @param nombre el nombre del titular.
102      */
103     public void setNombre(String nombre) {
104         this.nombre = nombre;
105     }
106 }

```

```
106
107 /**
108  * Obtiene el número de la cuenta bancaria.
109  *
110  * @return el número de la cuenta.
111  */
112 public String getCuenta() {
113     return cuenta;
114 }
115
116 /**
117  * Establece el número de la cuenta bancaria.
118  *
119  * @param cuenta el número de la cuenta.
120  */
121 public void setCuenta(String cuenta) {
122     this.cuenta = cuenta;
123 }
124
125 /**
126  * Obtiene el saldo actual de la cuenta.
127  *
128  * @return el saldo actual.
129  */
130 public double getSaldo() {
131     return saldo;
132 }
133
134 /**
135  * Establece el saldo de la cuenta.
136  *
137  * @param saldo el saldo de la cuenta.
138  */
139 public void setSaldo(double saldo) {
140     this.saldo = saldo;
141 }
142
143 /**
144  * Obtiene el tipo de interés de la cuenta.
145  *
146  * @return el tipo de interés.
147  */
148 public double getTipoInterés() {
149     return tipoInterés;
150 }
151
152 /**
153  * Establece el tipo de interés de la cuenta.
154  *
155  * @param tipoInterés el tipo de interés.
156  */
157 public void setTipoInterés(double tipoInterés) {
158     this.tipoInterés = tipoInterés;
159 }
160 }
161
```


2. Generar documentación JavaDoc para todo el proyecto y comprueba que abarca todos los métodos y atributos de la clase Ccuenta. Poner textos descriptivos de la clase y de los métodos. Utilizar las etiquetas más comunes de Javadoc: @author, @version, @since, @see (ejemplo: “@see Java Dcoumentation”), @param y @return.

Index de la documentacion:

PACKAGECLASSUSETREINDEXHELP

PACKAGE: DESCRIPTION | RELATED PACKAGES | CLASSES AND INTERFACES

SEARCH Search

Package com.mycompany.cuentas

package com.mycompany.cuentas

Classes

Class	Description
CCuenta	Clase que representa una cuenta bancaria.
Main	Clase principal que contiene el método main para ejecutar la aplicación de gestión de cuentas bancarias.

Copyright © 2024. All rights reserved.

Clase Main:

PACKAGECLASSUSETREINDEXHELP

SUMMARY: NESTED | FIELD | CONSTR | METHODDETAIL: FIELD | CONSTR | METHOD

SEARCH Search

Package com.mycompany.cuentas

Class Main

java.lang.Object[Ⓜ]
com.mycompany.cuentas.Main

public class Main
extends Object[Ⓜ]

Clase principal que contiene el método main para ejecutar la aplicación de gestión de cuentas bancarias.

Since:
2024-05-24

Version:
1.0

Author:
Abraham Quintana

See Also:
Java Documentation[Ⓜ]

Constructor Summary

Constructors

Constructor	Description
Main()	

Method Summary

All MethodsStatic MethodsConcrete Methods

Modifier and Type	Method	Description
static void	main(String [Ⓜ] [] args)	Método principal que ejecuta la aplicación.

Methods inherited from class java.lang.Object[Ⓜ]
clone[Ⓜ], equals[Ⓜ], finalize[Ⓜ], getClass[Ⓜ], hashCode[Ⓜ], notify[Ⓜ], notifyAll[Ⓜ], toString[Ⓜ], wait[Ⓜ], wait[Ⓜ], wait[Ⓜ]

Constructor Details

Main

public Main()

Method Details

main

public static void main(String[Ⓜ][] args)

Método principal que ejecuta la aplicación.

Parameters:
args - los argumentos de la línea de comandos (no utilizados en esta aplicación).

Copyright © 2024. All rights reserved.

Clase CCuenta:

PACKAGECLASSUSE TREE INDEX HELP

SUMMARY NESTED | FIELD | CONSTR | METHODDETAIL: FIELD | CONSTR | METHOD

SEARCH 🔍 Search

X

Package com.mycompany.cuentas

Class CCuenta

java.lang.Object[®]
com.mycompany.cuentas.CCuenta

public class CCuenta
extends Object[®]

Clase que representa una cuenta bancaria.

Esta clase permite realizar operaciones básicas de una cuenta bancaria como ingresar y retirar dinero, así como consultar el saldo actual.

Since:
2024-05-24

Version:
1.0

Author:
Abraham Quintana

See Also:

Java Documentation[®]

Constructor Summary

Constructors

Constructor	Description
CCuenta()	Constructor por defecto.
CCuenta(String [®] nom, String [®] cue, double sal, double tipo)	Constructor con parámetros para inicializar una cuenta bancaria.

Method Summary

All MethodsInstance MethodsConcrete Methods:

Modifier and Type	Method	Description
double	estado()	Método que devuelve el saldo actual de la cuenta.
String [®]	getCuenta()	Obtiene el número de la cuenta bancaria.
String [®]	getNombre()	Obtiene el nombre del titular de la cuenta.
double	getSaldo()	Obtiene el saldo actual de la cuenta.
double	getTipoInterés()	Obtiene el tipo de interés de la cuenta.
void	ingresar(double cantidad)	Método para ingresar una cantidad de dinero en la cuenta.
void	retirar(double cantidad)	Método para retirar una cantidad de dinero de la cuenta.
void	setCuenta(String [®] cuenta)	Establece el número de la cuenta bancaria.
void	setNombre(String [®] nombre)	Establece el nombre del titular de la cuenta.
void	setSaldo(double saldo)	Establece el saldo de la cuenta.
void	setTipoInterés(double tipoInterés)	Establece el tipo de interés de la cuenta.

Methods inherited from class java.lang.Object[®]
clone[®], equals[®], finalize[®], getClass[®], hashCode[®], notify[®], notifyAll[®], toString[®], wait[®], wait[®], wait[®]

Constructor Details

CCuenta

public CCuenta()
Constructor por defecto.

CCuenta

public CCuenta(String[®] nom,
String[®] cue,
double sal,
double tipo)
Constructor con parámetros para inicializar una cuenta bancaria.

Parameters:
nom - el nombre del titular de la cuenta.
cue - el número de la cuenta bancaria.
sal - el saldo inicial de la cuenta.
tipo - el tipo de interés de la cuenta.

Method Details

estado

public double estado()
Método que devuelve el saldo actual de la cuenta.
Returns:
el saldo actual de la cuenta.

retirar

public void retirar(double cantidad)
Método para retirar una cantidad de dinero de la cuenta.
Parameters:
cantidad - la cantidad a retirar.

ingresar

public void ingresar(double cantidad)
Método para ingresar una cantidad de dinero en la cuenta.
Parameters:
cantidad - la cantidad a ingresar.

getNombre

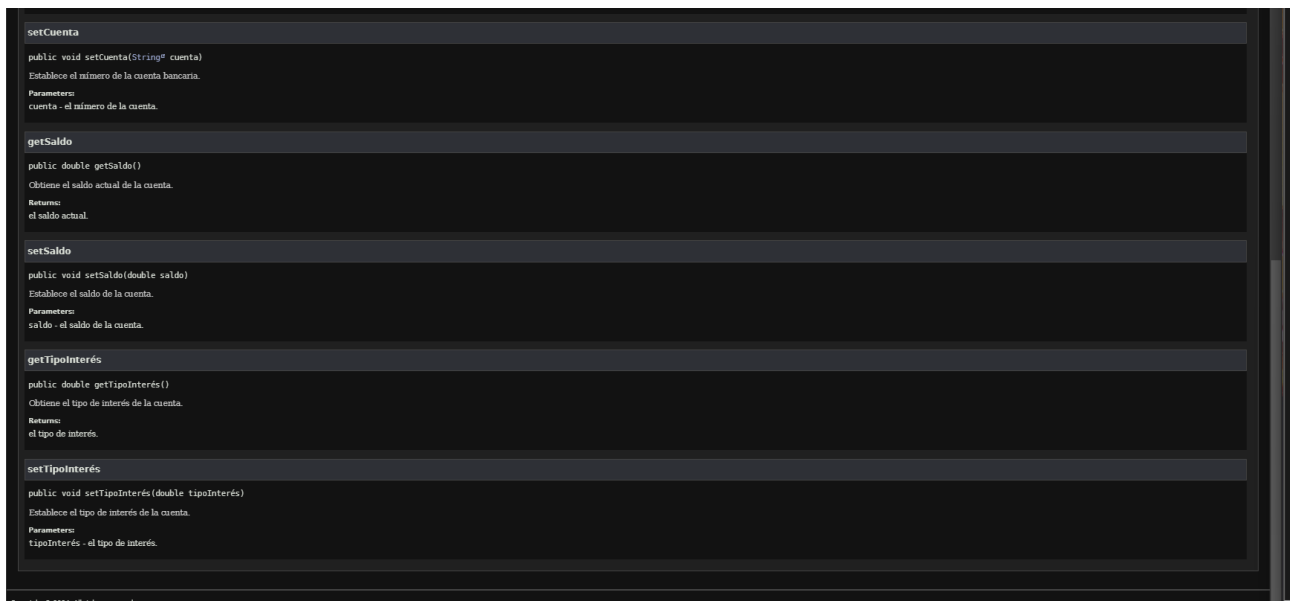
public String[®] getNombre()
Obtiene el nombre del titular de la cuenta.
Returns:
el nombre del titular.

setNombre

public void setNombre(String[®] nombre)
Establece el nombre del titular de la cuenta.
Parameter:
nombre - el nombre del titular.

getCuenta

public String[®] getCuenta()
Obtiene el número de la cuenta bancaria.
Returns:
el número de la cuenta.



Conclusión:

La refactorización es la manera de dejar un código mucho más limpio, legible y facilita el mantenimiento a nosotros mismos y a cualquier otro programador que tenga que añadir/modificar funcionalidades. Además todos los IDE que he visto hasta ahora proporcionan herramientas que facilitan enormemente ese trabajo al que muchas veces no se da la importancia que merece.

Git es el sistema de control de versiones mas utilizado, realmente es el único que he usado, aunque en mi empresa usan subversion para la primera versión del ERP con java 1.5, claramente GIT añade mucha funcionalidad y facilita enormemente el control de cambios realizados. Considero que es una herramienta que cualquier desarrollador debe conocer y utilizar a día de hoy. Además igual que pasa con la refactorización, en todos los IDE que he visto, existen herramientas que facilitan mucho su uso para desarrolladores que no conozcan sus comandos. Personalmente uso tanto las herramientas del IDE como los comandos según me sea más cómodo.

En cuando al JAVADOC, lo he visto implementado en todos los proyectos en los que he trabajado hasta hoy. Bien es cierto que no se suele generar los ficheros HTML para visionarlo en navegadores web, ya que simplemente poniendo el puntero del ratón en la llamada de algún método o instanciación de clases, los IDE te muestran la información de ese método o clase concreta en una pequeña ventana sin tener que abrir un navegador y estar buscando el elemento concreto que queremos consultar, que al final es más cómodo y rápido.

Finalmente ya que no me cuesta nada, voy a commitear el código resultante incluyendo los comentarios/javadoc y a subirlo todo a mi repositorio. Aunque la documentación generada no se va a subir ya que en mi .gitignore, está añadida la regla de no añadir todo lo que está en la carpeta target.